

- تحلیل ساده و احدهای responsive با استفاده از تغییرات نسبت به حالت baseline
- ② تحلیل robust بر اساس تفاوت پاسخ محرک و راس
- اعتبار منحنی . Pop. مطابق مقادیر pseudo-trial

① کدام نوبت ها محرک است  
Stim onset تغییرات منحنی →

② برای هر نوبت responsive باید در محدوده 70 تا 40 درصد باشد  
تولید نم

③ ترتیب نوبت منطقی هست  
→

نوازن چه زمانی محرک است - stim-on پاسخ ی در

پاسخ —  $\text{response}(t) = | \text{PSTH}(t) - \text{baseline} |$  (vs. latency)

① فعالیت نوازن چه زمانی نسبت به قبل از محرک تغییر می کند؟

پاسخ —  $\text{response}(t) = | \text{PSTH}_{\text{left}}(t) - \text{PSTH}_{\text{right}}(t) |$  (side-selective latency)

② فعالیت نوازن چه زمانی اطلاعاتی درباره ی چه استیون محرک دارد؟

② نه robust پاسخ سوال دوم را می دهد!

```
def q3_paper_shuffled_labels(labels, strata, n_shuffles, rng):
    """Shuffle labels within each stratum while preserving label counts."""
```

```
    pseudo = np.zeros((n_shuffles, len(labels)), dtype=np.float32)
    rows = np.arange(n_shuffles)[: , None]
```

```
    for stratum in np.unique(strata):
```

tricks of on certain stratum

```
        trial_index = np.flatnonzero(strata == stratum)
        n_left = int(labels[trial_index].sum())
        if n_left == 0:
            continue
        if n_left == len(trial_index):
            pseudo[:, trial_index] = 1
            continue
```

full left or full right  
reverses don't shuffle

```
        scores = rng.random((n_shuffles, len(trial_index)))
```

```
        selected = np.argpartition(
```

```
            scores, n_left - 1, axis=1
        )[:, :n_left]
```

```
        pseudo[rows, trial_index[selected]] = 1
```

→ from each row pick k-smallest!  
→ flipping

```
    return pseudo
```

converting to real indices

| trial index | prior block | choice | stimulus side | labels | strata |
|-------------|-------------|--------|---------------|--------|--------|
| 0           | left        | left   | left          | 1      | A      |
| 1           | left        | right  | right         | 0      | B      |
| 2           | left        | left   | right         | 0      | A      |
| 3           | right       | right  | left          | 1      | C      |
| 4           | right       | right  | right         | 0      | C      |
| 5           | right       | left   | left          | 1      | D      |

← stratum

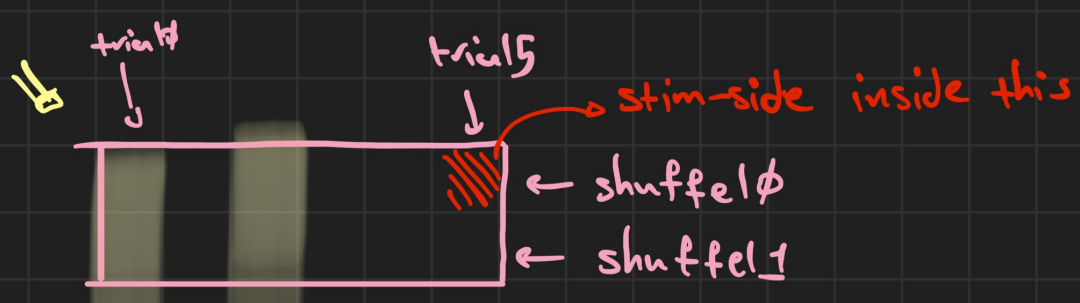
↓  
 1 for left stim-side  
 0 for right stim-side

| Prior | choice | strate |
|-------|--------|--------|
| L     | L      | A      |
| L     | R      | B      |
| R     | R      | C      |
| R     | L      | D      |

labels = np.array [ 1, 0, 0, 1, 0, 1 ]  
 strate = np.array [ A, B, A, C, C, D ]

shuffle = 2

Pseudo



for A  $\rightarrow$  trial-index =  $[0, 2]$

for A  $\rightarrow$  labels [trial-index] = labels  $[0, 2]$   
how many trials in A =  $[1, 0]$

Scores 

|     |     |
|-----|-----|
| 0/0 | 0/2 |
| 0/1 | 0/7 |

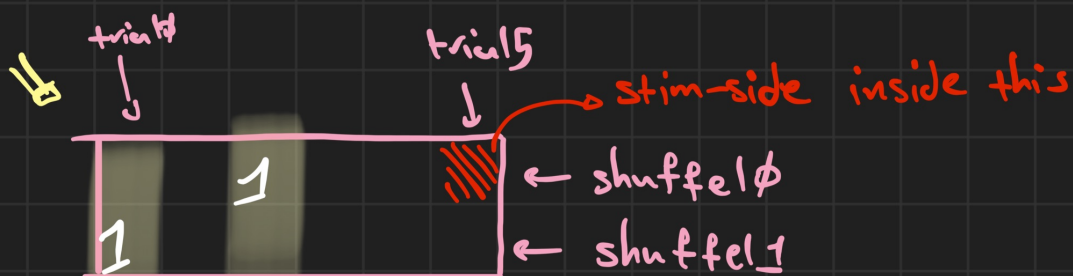
 $\left\{ \begin{array}{l} \text{how many shuffles!} \end{array} \right.$

$\rightarrow$  local-column 0  $\rightarrow$  real column 0

$\rightarrow$  local-column 1  $\rightarrow$  real column 2

we don't want to change the number of 1's in Strat.

\* so we choose the lowest score of each row and give it a 1 (if there were two one's we'd choose 2 lowest)



there are cond's where we don't interpolate.

handling the case where crossing doesn't happen is imp.

crossing  $[0]$  is index  $\rightarrow$  of first crossing



```

def q3_paper_latency(times, distance, fraction=0.70):
    """First 70%-of-amplitude crossing, interpolated on the 10-ms grid."""

    threshold = distance.min() + fraction * np.ptp(distance)
    crossing = np.flatnonzero(distance >= threshold) → binary_mask for crossings
    if len(crossing) == 0:
        return np.nan

    index = int(crossing[0])
    if index == 0 or distance[index] == distance[index - 1]: } if - no - crossing
        return times[index] * 1000

    weight = (threshold - distance[index - 1]) / (
        distance[index] - distance[index - 1]
    ) } interpol.

    return (
        times[index - 1] + weight * (times[index] - times[index - 1])
    ) * 1000 } turn index - to time

```

Python

```
times = np.array([0.00, 0.01, 0.02, 0.03])
distance = np.array([2.0, 2.5, 5.0, 8.0])
threshold = 6.2
```

بین این دو نقطه قرار دارد:

Plain text

time = 0.02 → distance = 5.0  
time = 0.03 → distance = 8.0

$$\text{thresh.} \Rightarrow 2.0 + 0.7 (8.0 - 2.0)$$
$$2.0 + 4.2 = 6.2$$

time of crossing is b.w. 20 and 30 ms but

when exactly!

interpol.

| $\text{dis}[\text{index}-1]$ |     | $\text{dis}[\text{index}]$ |
|------------------------------|-----|----------------------------|
| 5.0                          | 6.2 | 8.0                        |
| 20                           | 2   | 30                         |

$\Rightarrow \tau = 24 \text{ ms!}$

|                                                             |                                               |
|-------------------------------------------------------------|-----------------------------------------------|
| $\begin{array}{r} 1.2 \\ 6.2 - 5 \\ \hline 3.0 \end{array}$ | $\begin{array}{r} ? \\ \hline 10 \end{array}$ |
|-------------------------------------------------------------|-----------------------------------------------|

$\Rightarrow ? = 4$

$\hookrightarrow \text{time}[\text{index}] - \text{time}[\text{index}-1]$

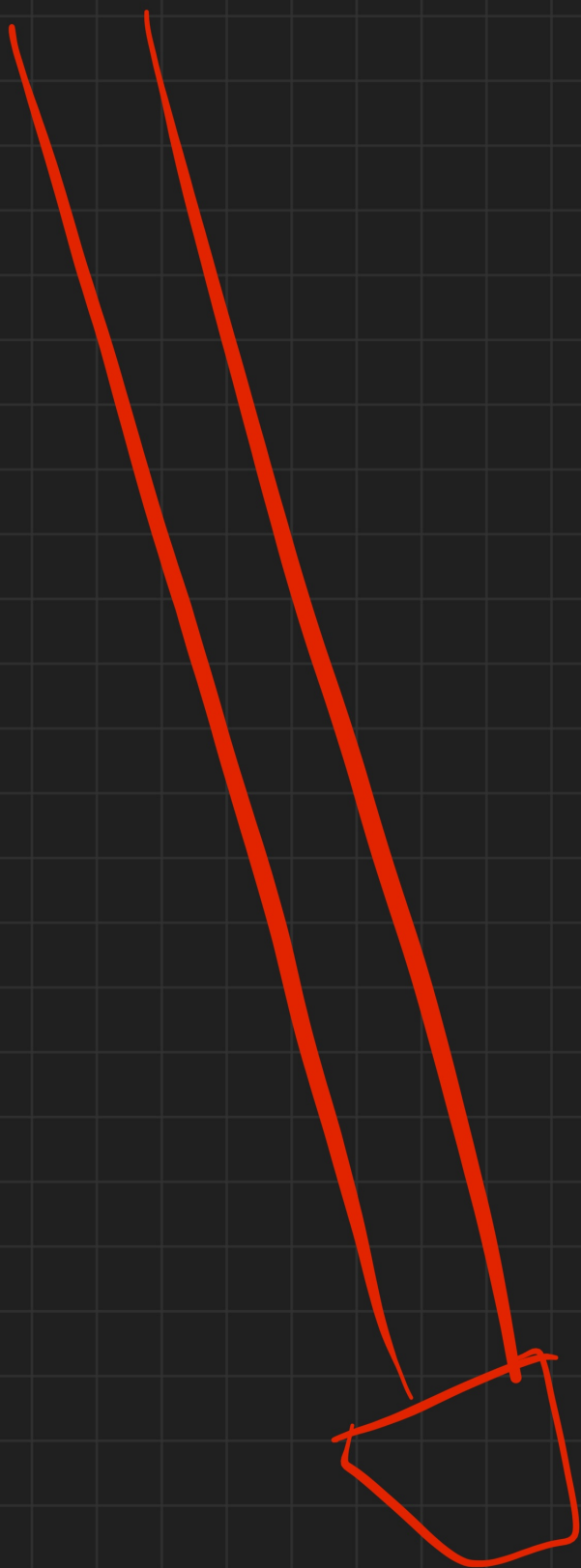
weight

```
q3_paper_region_to_group = {  
    region: group  
    for group, regions in q3_anatomical_regions.items()  
    for region in regions  
}  
  
q3_paper_window = (  
    (q3_meta.times >= 0) & (q3_meta.times <= 0.15)  
)  
  
q3_paper_times = q3_meta.times[q3_paper_window]  
q3_paper_rng = np.random.default_rng(Q3_PAPER_SEED)
```

} → from group → region  
to region → group

} → bin-mask

} → actual time bi



Python



```
q3_anatomical_regions = {  
    "visual": ["VISp", "VISl"],  
    "motor": ["MOp", "MOs"]  
}
```

for group, regions in

## اجرای مرحله به مرحله

دیکشنری اولیه:

Python



```
q3_anatomical_regions = {  
    "visual": ["VISp", "VISl"],  
    "motor": ["MOp", "MOs"]  
}
```

## دور اول حلقه‌ی بیرونی

Python



```
group = "visual"  
regions = ["VISp", "VISl"]
```

for region in regions → region ∈ group  
"VISp" ∈ visual

→ create a time-series array

```
q3_true_sumsq = { 1 } bin-mask  
    region: np.zeros(q3_paper_window.sum())  
    for region in q3_paper_region_to_group
```

} true sum - squarved!

```
q3_null_sumsq = {  
    region: np.zeros((Q3_PAPER_SHUFFLES, q3_paper_window.sum()))  
    for region in q3_paper_region_to_group
```

} trials!

} num. neurons. with keys from:

```
q3_paper_n_units = dict.fromkeys(q3_paper_region_to_group, 0)
```

fill with 0

```
q3_paper_pids = {  
    region: set() for region in q3_paper_region_to_group
```

}  $\hookrightarrow$  region  $\rightarrow$  set of pids (unique)  $\hookrightarrow$  remember last page??

```
q3_paper_target_pids = q3_meta.clusters.loc[  
    q3_meta.clusters['acronym'].isin(q3_paper_region_to_group), 'pid'
```

```
].dropna().unique()  $\rightarrow$  only keep uniques!
```

go to clusters table look at acronym  
if it's relevant then copy its pid columns  
all of their



q3 - true - sumSq = {

"vis\_p" : [0.0, 0.0, 0.0, 0.0],

"vis\_l" : [0.0, 0.0, 0.0, 0.0],

"map" : [0.0, 0.0, 0.0, 0.0]

}

q3 - null - sumSq = {

"vis\_p" :  shuffles (1000)

"vis\_l" :  how many time\_bin in time win

"map" :

}

```
q3_paper_n_units = dict.fromkeys(q3_paper_region_to_group, 0)
```

```
q3_paper_pids = {
```

```
    region: set() for region in q3_paper_region_to_group
```

```
}
```

```
q3_paper_target_pids = q3_meta.clusters.loc[
```

```
    q3_meta.clusters['acronym'].isin(q3_paper_region_to_group), 'pid'
```

```
].dropna().unique()
```



drop NA's

keep unique

row\_sel

col\_sel

Picking all pids for a certain region

q3 - n - units = 1

"vis\_p" : 0

"vis\_l" : 0

"map" : 0

}

q3 - target\_pid = [

pid's for said region]

```
for pid in tqdm.tqdm(  
    q3_paper_target_pids, desc='Paper-aligned Q3 trajectory'  
):
```

```
    psth_pid, clusters_pid, trials_pid = get_psth_for_insertion(  
        pid, q3_meta
```

```
    )  
    target_units = clusters_pid['acronym'].isin(  
        q3_paper_region_to_group
```

```
    ).to_numpy()  
    if not target_units.any():  
        continue
```

```
    left_contrast = trials_pid['contrastLeft'].to_numpy(dtype=float)
```

```
    right_contrast = trials_pid['contrastRight'].to_numpy(dtype=float)
```

```
    choice = trials_pid['choice'].to_numpy(dtype=float)
```

```
    block = trials_pid['probabilityLeft'].to_numpy(dtype=float)
```

```
# The paper code uses every assigned stimulus side, including zero  
# contrast; side is represented by which contrast column is non-NaN.
```

```
left_label = np.isfinite(left_contrast)
```

```
right_label = np.isfinite(right_contrast)
```

```
keep = (  
    (left_label | right_label) &  
    np.isfinite(choice) &  
    np.isfinite(block)
```

```
)  
labels = left_label[keep]
```

```
if not labels.any() or labels.all():  
    continue
```

} → data for pid

} → which neurons to target!

} → if no neuron found skip pid

} → pulling data from trials & meta

} → encoding Stim-side

} → left = 1, right = 0 in labels!

binary mask!  
 → is the trial valid?  
 it should have some contrast value?  
 it should also have a valid choice and block value

```
strata = pd.factorize(pd.MultiIndex.from_arrays([
    block[keep] == 0.8,
    choice[keep] == 1,
]))[0]
```

↳ creates a pair of (block, choice) for each trial

```
pseudo = q3_paper_shuffled_labels(
    labels, strata, Q3_PAPER_SHUFFLES, q3_paper_rng
)
```

↳ list of possible block x choice  
 ↳ list of left-right sides (side-info)

```
pseudo_left_n = pseudo.sum(axis=1)
pseudo_right_n = len(labels) - pseudo_left_n
if np.any(pseudo_left_n == 0) or np.any(pseudo_right_n == 0):
    continue
```

↳ all left  
 ↳ all right  
 ↳ cancel  
 trials worth keeping  
 ↳ neurons of relevant region

```
values = psth_pid[keep][:, target_units, :][:, :, q3_paper_window]
values = values.astype(np.float32)
```

↳ imp. time window

```
target_clusters = clusters_pid.loc[target_units].reset_index(drop=True)
```

neuron - info  
 ↳ row of target units (neurons of relevant region)

| trial.n | block cond. | choice cond. |   |
|---------|-------------|--------------|---|
| ∅       | T           | T            | ∅ |
| 1       | T           | F            | 1 |
| 2       | T           | T            | ∅ |
| 3       | f           | F            | 2 |
| 4       | f           | F            | 2 |
| 5       | f           | T            | 3 |

∅ → strata  
 1 → smth. else  
 ⇒ [∅] → [∅, 1, 2, 3] → array

values.shape = (6, 4, 5)

target cluster  
time bins!  
valid trials

Contains left labels → T, F, T, F, T, F

from unit(cluster metadata) 0, 2, 4 left trials

```
for region, region_units in target_clusters.groupby('acronym'):
    region_index = region_units.index.to_numpy()
    region_values = values[:, region_index, :]
    # path for certain neurons
    true_difference = (
        region_values[labels].mean(axis=0) -
        region_values[~labels].mean(axis=0)
    )
    q3_true_sumsq[region] += np.square(true_difference).sum(axis=0)

    flat_values = region_values.reshape(len(labels), -1)
    pseudo_left = (
        pseudo @ flat_values
    ) / pseudo_left_n[:, None]
    pseudo_right = (
        (1 - pseudo) @ flat_values
    ) / pseudo_right_n[:, None]
    pseudo_difference = (pseudo_left - pseudo_right).reshape(
        Q3_PAPER_SHUFFLES,
        region_values.shape[1],
        region_values.shape[2]
    )
    q3_null_sumsq[region] += np.square(
        pseudo_difference
    ).sum(axis=1)

    q3_paper_n_units[region] += region_values.shape[1]
    q3_paper_pids[region].add(pid)
```

region unit  
ISP  
MOP  
STS  
region

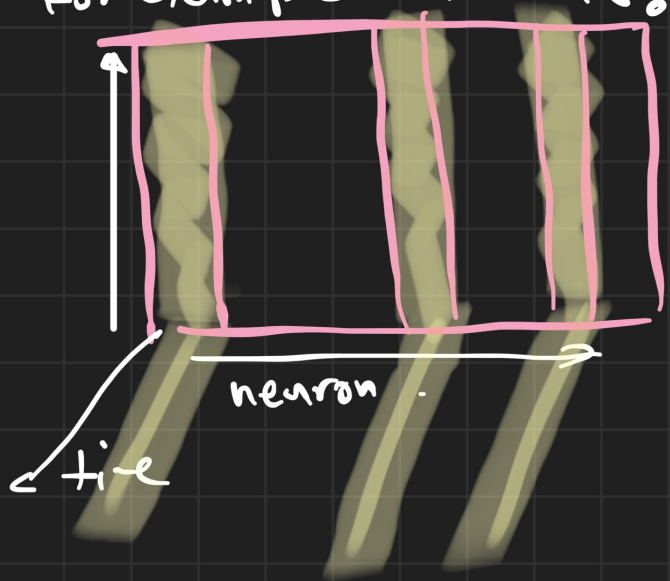
trial averaging on all left trials for a certain neuron  
right

certain time bins we already selected so its about 10  
numbers per trials

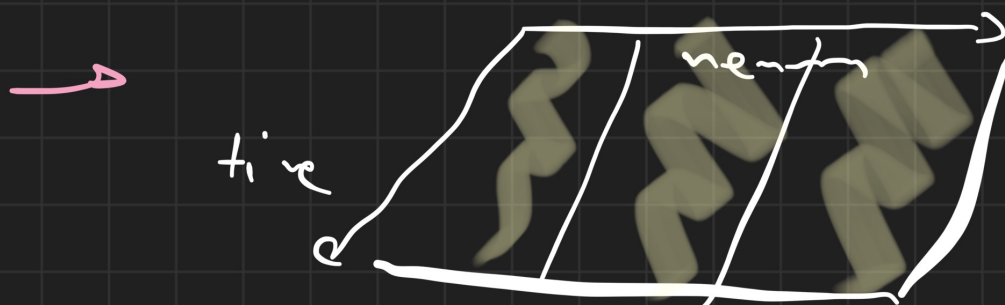
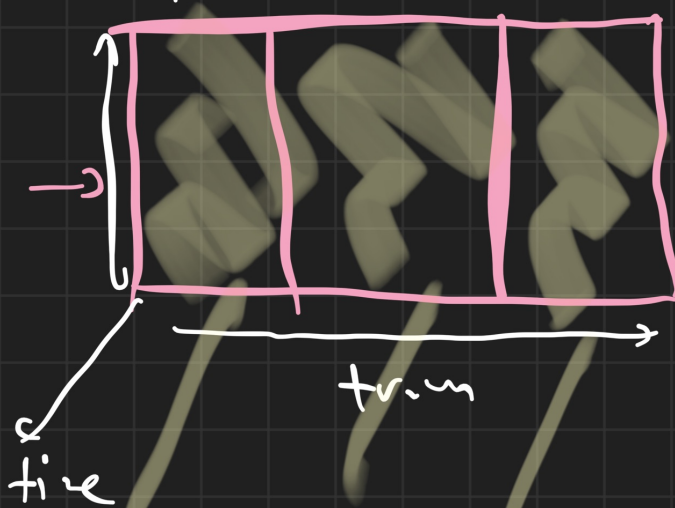




for example left trials!



trials



```
for region, region_units in target_clusters.groupby('acronym'):
    region_index = region_units.index.to_numpy()
    region_values = values[:, region_index, :]
```

```
true_difference = (
    region_values[labels].mean(axis=0) -
    region_values[~labels].mean(axis=0)
)
```

```
q3_true_sumsq[region] += np.square(true_difference).sum(axis=0)
```

each PSD contributes a wave form

```
flat_values = region_values.reshape(len(labels), -1)
```

```
pseudo_left = (
    pseudo @ flat_values
) / pseudo_left_n[:, None]
```

```
pseudo_right = (
    (1 - pseudo) @ flat_values
) / pseudo_right_n[:, None]
```

```
pseudo_difference = (pseudo_left - pseudo_right).reshape(
    Q3_PAPER_SHUFFLES,
    region_values.shape[1],
    region_values.shape[2]
)
```

```
q3_null_sumsq[region] += np.square(
    pseudo_difference
).sum(axis=1)
```

each probe cont. subtractions & waveforms

```
q3_paper_n_units[region] += region_values.shape[1] → neuron counts
q3_paper_pids[region].add(pid) → adding PSD
```

region-values.shape == (4, 2, 3)<sup>t n b</sup>

[ trial = 0  
→ [[10, 11, 12], [20, 21, 22]]  
[[30, 31, 32], [40, 41, 42]] → trial 1  
[[50, 51, 52], [60, 61, 62]]  
[[70, 71, 72], [80, 81, 82]]  
]

↓

after reshape

flat-values [ trial 0  
[10, 11, 12, 20, 21, 22],  
[30, 31, 32, 40, 41, 42], → trial 1  
[50, 51, 52, 60, 61, 62],  
[70, 71, 72, 80, 81, 82]  
]

"you!" ⇒ one row one trial!

pseudo = [ ⇒ shape (2, 4)  
[1, 0, 1, 0],  
[1, 1, 0, 0]  
]

pseudo left → trial averaging through mat. mult.

Summing trials 0 and 2

Summing trials 0 and 1

↓

pseudo @ flat-values

[60, 62, 64, 80, 82, 84]  
[40, 42, 44, 60, 62, 64]



dividing by the cnt. of left and right trials

using norm ✓

region values contains region converted psth for pD

$t$   $n$   $b$   
↓  $n$   $n$   $n$   
or shuffles  $n$ -averaging  
used to turn back to original after getting the diff.

```
q3_paper_rows = []  
✓ for region in q3_paper_region_to_group:  
    n_units = q3_paper_n_units[region]  
    ✓ if n_units < Q3_PAPER_MIN_UNITS:  
        continue  
  
    distance = np.sqrt(q3_true_sumsq[region] / n_units)  
    null_distance = np.sqrt(q3_null_sumsq[region] / n_units)  
    amplitude = np.ptp(distance)  
    null_amplitude = np.ptp(null_distance, axis=1)  
    p_value = (  
        1 + np.sum(null_amplitude >= amplitude)  
    ) / (Q3_PAPER_SHUFFLES + 1)  
  
    ✓ q3_paper_rows.append({  
        'acronym': region,  
        'anatomical_group': q3_paper_region_to_group[region],  
        'n_pids': len(q3_paper_pids[region]),  
        'n_units': n_units,  
        'amplitude_hz': amplitude,  
        'latency_ms': q3_paper_latency(  
            q3_paper_times, distance, Q3_ONSET_FRACTION  
        ),  
        'p_value': p_value  
    })
```

$\sqrt{\frac{\text{sum sq}}{\text{neuron}}}$   neurons  
 $\sqrt{\frac{\text{sum sq}}{\text{neuron}}}$  neurons  
null  $\geq$  real  
 $\frac{1+n}{1+m}$   
total shuffles  
p. ins + pencil 66

هـ

region

اضافه کردن ردیف برای هر

```
q3_paper_regions = pd.DataFrame(q3_paper_rows)
q3_paper_regions['q_value'] = q3_bh_fdr(
    q3_paper_regions['p_value'].to_numpy() → all p-values in order
)
q3_paper_regions['significant'] = (
    q3_paper_regions['q_value'] < Q3_PAPER_FDR_ALPHA → defining sig.
)
q3_paper_significant_regions = q3_paper_regions[
    q3_paper_regions['significant']
].copy() → making a list of all sig regions with their data
q3_paper_group_summary = (
    q3_paper_significant_regions
    .groupby('anatomical_group')
    .agg(
        n_regions=('acronym', 'nunique'),
        n_units=('n_units', 'sum'), → summing all units
        latency_ms=('latency_ms', 'median'), → finding the median
        q25_ms=('latency_ms', lambda x: x.quantile(0.25)), → q1 lat.
        q75_ms=('latency_ms', lambda x: x.quantile(0.75)) → q3 lat.
    )
    .sort_values('latency_ms')
) → b.w. all regions in gr.
```

| acronym | anatomical_group      | n_units | latency_ms |
|---------|-----------------------|---------|------------|
| NOT     | Visual midbrain       | 92      | 45         |
| OP      | Visual midbrain       | 39      | 49         |
| APN     | Visual midbrain       | 1543    | 119        |
| VISp    | Primary visual cortex | 880     | 50         |

agg ( 'old\_column\_name', function )  
=  
new\_column\_name

visual-midbrain → not  
OP  
APN  
prim-visual-cortex → VISP

rest is just plotting and  
cosmetics!